

ĐỐI TƯỢNG PARAMETER

Sử dụng đối tượng Parameter để tạo ra các tham số và truyền vào giá trị cho các tham số trong thủ tục Store procedure của đối tượng SqlCommand, hoặc các tham số trong báo biểu.

Tạo và truyền vào giá trị cho các đối số của store

- Khi gọi thực thi một store bằng đối tượng SqlCommand thì chúng ta cần phải tạo và truyền vào giá trị cho store.
- Đối tượng sử dụng để truyền vào cho các store là đối tượng Parameter.

Phương thức khởi tạo của đối tượng Parameter.

SqlParameter p=new SqlParameter(tenDoiSo, giatri);

- tenDoiSo: là tên đối số trong Store procedure
- giaTri: Là giá trị mà đối số trong Store procedure nhận được.

Phương thức khởi tạo của đối tượng Parameter.

SqlParameter p=new SqlParameter(tenDoiSo, giatri);

- tenDoiSo kiểu String: là tên đối số trong Store procedure
- giaTri kiểu object: Là giá trị mà đối số trong Store procedure nhận được.

SqlParameter p=new SqlParameter(tenDoiSo, kieuDuLieu, doRong);

- kieuDuLieu: là kiểu dữ liệu của đối số nằm trong enum SqlDbType
- Độ rộng là độ rộng kiểu dữ liệu

Các bước tạo và truyền vào các parameter để thực thi store

B1. Tạo đối tượng SqlCommand(tenstore,con)

B2. Gán thuộc tính CommandType= CommandType.StoredProcedure;

B3. Tạo ra đối tượng Parameter cho từng đối số của Store và truyền vào giá trị.

B4. Add parameter vào tập hợp Parameters của đối tượng SqlCommand

B5. Lặp lại B3 và B4 cho đến khi hết đối số của Store.

Ví dụ: Gọi Store strNhapSV

```
SqlCommand dc=new SqlCommand("strNhapSV",con);  
dc.CommandType = CommandType.StoredProcedure;  
SqlParameter p;  
p = new SqlParameter("@malop", OleDbType.VarChar, 20);  
p.Value = "01.1";  
dc.Parameters.Add(p);  
p = new OleDbParameter("@MaSv", "01.1.190");  
dc.Parameters.Add(p);  
p = new OleDbParameter("@TenSV", "Chu van quenh");  
dc.Parameters.Add(p);  
dc.ExecuteNonQuery();
```

Xây dựng phương thức gọi một store *hành động* bất kỳ.

```
Public void StoreNonParam(string tenStore, params object[]  
DoiSo_giatri)  
    { try{ MoKetNoi();  
        cm = new SqlCommand(tenStore, con);  
        cm.CommandType = CommandType.StoredProcedure;  
        SqlParameter p;  
        for (int i = 0; i < DoiSo_giatri.Length /2; i+=2)  
        {  
p = new SqlParameter(DoiSo_giatri[i].ToString(), DoiSo_giatri[i + 1]);  
        cm.Parameters.Add (p);  
        }cm.ExecuteNonQuery(); }  
        catch( Exception ex) { throw new Exception(ex.message);  
        }
```

Xây dựng phương thức gọi một store *select* bất kỳ.

```
Public SqlDataReader StoreReaderParam(string tenStore, params
object[] DoiSo_giatri)
{ try{ MoKetNoi();
    cm = new SqlCommand(tenStore, con);
    cm.CommandType = CommandType.StoredProcedure;
    SqlParameter p;
    for (int i = 0; i < DoiSo_giatri.Length /2; i+=2)
    {
p = new SqlParameter(DoiSo_giatri[i].ToString(), DoiSo_giatri[i + 1]);
    cm.Parameters.Add (p);
    } return cm.ExecuteReader (); }
    catch( Exception ex) { throw new Exception(ex.message);
    }
```

Khai báo và sử dụng phương thức gọi store.

```
static void Main()
{ try {
    KetNoi kn = new KetNoi("MinhHue", "QITV");
    SqlDataReader dr=kn.StoreReaderParam("strSelectSV",
"@malop", "201081");
    kn.StoreNonParam("strNhapSv", "@Malop", "201081",
"@MaSV", "sv002", "@TenSV", "tran tuong bo", "@diem", 9);
} catch (Exception ex)
{ MessageBox.Show(ex.Message); }
}
```

Phương thức tính DeriveParameters của lớp SqlCommandbuilder

Phương thức này dùng để tự động lấy về các Parameter tương ứng với Store và add vào tập hợp Parameter của đối tượng SqlCommand. Vì vậy khi đó ta không cần phải truyền vào tạo ra các Parameter tương ứng với các đối số của store. Mà chỉ cần duyệt qua các Parameter và truyền vào giá trị cho các parameter.

Các bước gọi thực thi store có sử dụng phương thức tĩnh `DeriveParameters`

B1. Tạo đối tượng `SqlCommand(tenstore, con)`

B2. Gán thuộc tính `CommandType= CommandType.StoredProcedure;`

B3. Gọi phương thức tĩnh **`SqlCommandbuilder . DeriveParameters(cm)`** để lấy về các parameter.

B4. Duyệt qua tập hợp các Parameter (từ chỉ số 1- hết) của đối tượng `SqlCommand` và gán giá trị cho các parameter.

B5. Gọi phương thức `ExcuteXXX()` để thực thi.

VD: Xây dựng phương thức thực thi store select.

```
public DataTable ExcuteSelectStore(string tenSP, params
object[] giatri)
{
    Open();
    SqlCommand cm = new SqlCommand(tenSP, con);
    cm.CommandType = CommandType.StoredProcedure;
    //Lấy về các Parameter
    SqlCommandBuilder.DeriveParameters(cm);
    for (int i = 1; i < cm.Parameters.Count; i++)
    {
        cm.Parameters[i].Value = giatri[i - 1];
    }
    SqlDataAdapter da = new SqlDataAdapter(cm);
    DataTable dt = new DataTable();
    da.Fill(dt);
    return dt;
}
```

VD: Xây dựng phương thức thực thi store hành động

```
public void ExcuteNonStore(string tenSP, params object[]  
giatri)  
{  
    Open();  
    //b1  
    SqlCommand cm = new SqlCommand(tenSP, con);  
    cm.CommandType = CommandType.StoredProcedure;  
    //Lấy về các Parameter  
    SqlCommandBuilder.DeriveParameters(cm);  
    for (int i = 1; i < cm.Parameters.Count; i = i + 1)  
    {  
        cm.Parameters[i].Value = giatri[i - 1];  
    }  
    cm.ExecuteNonQuery();  
}
```

VD: Gọi thực thi

```
static void Main()
```

```
{ try{
```

```
    KetNoi kn = new KetNoi("MinhHue", "QITV");
```

```
    SqlDataReader dr = kn.StoreReader("strSelectSV","201081");
```

```
    kn.StoreNonQuery("strNhapSv", "201081", "sv002", "A", 9);
```

```
}
```

```
catch (Exception ex) { MessageBox.Show(ex.Message); }
```

```
}
```